

AetherControl en Kotlin/Compose: del Programa .NET/Xamarin (TS6C3) a MVVM Reactivo con MQTT y Joystick de Baja Latencia

Andrés Felipe Martínez Henao

Programación Móvil TS6C3 – 3 créditos

Tecnología en Desarrollo de Software (5º sem) – Ing. Sistemas y Computación (6º sem)

Universidad Tecnológica de Pereira

F.martinez5@utp.edu.co

Proyecto AetherNet – Área 1 (App AetherControl)

IDs MOV-01..12 – docs/Programacion-Movil/ – Kotlin + Compose + Paho

Abstract—El programa TS6C3 está redactado sobre .NET/C#/Xamarin.Forms/XAML; AetherControl lo implementa con la pila nativa moderna Kotlin/Jetpack Compose preservando los mismos patrones. Se mapean U1 plataforma/lenguaje, U2 front declarativo (XAML→Compose), U3 back MVVM + emuladores, y U4 datos/hardware (JSON-REST→MQTT/WebSocket, Room, Bluetooth SPP). La app expone MVVM estricto con `DashboardViewModel` y `sealed interface DashboardUiState` (`Connected/Connecting/Disconnected/Error`), `StateFlow` + `coroutines`, `Room` y `DataStore`. El núcleo de comunicación es `MqttManager` (Eclipse Paho 1.2.5, `tcp://1883` + `ws://9001` fallback, ACL `aethernet/#`, `SharedFlow` para `rover/telemetry`, `access/event`, `seguridad/intrusion`), `ServiceLocator` DI manual y `AetherRepository`. El diferencial es el joystick virtual custom (Canvas 120 dp, `pointerInput/detectDragGestures`, normalización $X,Y \in [-1, 1]$, `deadband 60`, `throttle 50 ms/20 Hz`, `QoS0 aethernet/rover/command`) que logra <50 ms app→motor vía Gateway RF (99.6% TX 250→249). Al 2026-09-11: MOV-01..06 Done (assembleDebug + 16 tests), pendiente MOV-07 BT SPP y MOV-08..10 dashboard/tests.

Index Terms—Kotlin Jetpack Compose, MVVM `StateFlow`, Eclipse Paho MQTT, Room, joystick virtual, Bluetooth SPP, Retrofit

I. INTRODUCCIÓN: ADAPTACIÓN .NET→KOTLIN

El PDF 2361_6. TS6C3 enseña plataforma .NET, XAML, Pages/StackLayout/Grid/Views, Xamarin.Forms y MVVM. El PRD §4 excluye iOS/web y exige Android nativo; por tanto AetherControl traduce cada unidad a su equivalente Kotlin/Compose sin pérdida conceptual (Tab. I, docs/Programacion-Movil/programacion-movil.md).

II. MAPEO U1–U4 → IMPLEMENTACIÓN

A. U1 Plataforma y Lenguaje

Android Studio + Gradle KTS + JDK17 (CI `android-build`), Kotlin data classes (`RoverTelemetry`, `AccessEvent`) + sealed interfaces para estados exhaustivos (`DashboardUiState`), `coroutines`

Table I
EQUIVALENCIA TS6C3 (PDF) → AETHERNET

U	PDF (.NET/Xamarin)	AetherNet (Kotlin/Compose)
U1	Framework, IDE, C#	Android Studio, Gradle, Kotlin null-safety
U2	XAML, Pages, layouts	Compose composables, Column/Row/LazyGrid
U3	Xamarin MVVM, emuladores	ViewModel+StateFlow, AVD
U4	JSON-REST, permisos, HW	MQTT/WS, Room, Bluetooth SPP

(`viewModelScope.launch`, `Dispatchers.IO`), colecciones inmutables expuestas como `StateFlow`.

B. U2 Front Móvil (XAML→Compose)

Principio declarativo idéntico: se declara *qué* se ve. `Theme.kt` (Material3), `MainActivity` single-activity con `NavGraph`, `Column/Row/Box/modifiers` $\equiv$ `StackLayout/Grid`, `LazyVerticalGrid` para RF-1.1 Dashboard, controles: LED RGB local, joystick custom, telemetría. Throttling visual: recomposición solo en cambios de `uiState`.

C. U3 Back Móvil / MVVM

MVVM literal: `View (Compose)` $\xrightarrow{\text{collectAsState}}$ `uiState`
 $\leftarrow$ `DashboardViewModel` $\leftarrow$ `Repository` $\leftarrow$ `Room/MQTT`. `DashboardUiState` con 4 estados (`Connected(roverTelemetry, lastAccessEvent)`, `Disconnected(reason)`, `Connecting`, `Error(message)`) evita estados indefinidos. Puente `MutableStateFlow.asLiveData()` si se requiere. AVD para pruebas sin hardware.

D. U4 Datos y Hardware

Canal principal *no* es REST puro sino pub/sub MQTT (decisión IoT): topics JSON sobre `aethernet/#` (mismo payload que REST). `Retrofit/kotlinx.serialization` queda para request/response. Persistencia `Room` (`Entities.kt`, `Daos.kt`, `AppDatabase.kt`) como caché offline-first. Permisos runtime Android 12+:

BLUETOOTH_CONNECT, ACCESS_FINE_LOCATION (SPP $\neq$ BLE). Hardware remoto (rover/cerrojo) vía MQTT→ESP32; fallback BluetoothSocket SPP al HC-06 del nodo de acceso (RF-1.3, MOV-07) con UUID 00001101-..., loop en Dispatchers.IO.

III. MÉTODO: ARQUITECTURA Y COMUNICACIÓN

A. Estructura MVVM + Repository

ServiceLocator (DI manual) expone mqttManager derivado de getCurrentBaseUrl() (ServiceLocator.kt:114) + PreferencesManager DataStore con updateBaseUrl() dinámico + network_security_config cleartext para 192.168.1.14:8000 (MOV-01 f03190b). AetherRepository expone Flows: mqttConnectionState, roverTelemetryFlow, accessEventFlow; DashboardViewModel colecta push y hace fallback poll 5 s si Disconnected (MOV-03).

B. MQTT (MOV-03/11)

MqttManager Paho 1.2.5: conexión tcp://host:1883 con fallback ws://9001, usuario appbackend (ACL DEVOPS-02), SharedFlow para 4 topics (rover/telemetry:69, access/command:70, access/event:71, seguridad/intrusion:72, system/status:73), reconexión backoff, indicador UI MQTT ●/○/×. Verificado con mosquitto_sub -h 192.168.1.14 -t aethernet/# <50 ms (prd.md:50).

C. Joystick virtual (MOV-05/06)

JoystickScreen Canvas circular 120 dp + knob; pointerInput{detectDragGestures} → offset normalizado clampeado al radio, al soltar anima a (0,0) y emite centro exacto. JoystickMapper X,Y ∈ [-1,1] con dead-band ±0.15 (bajo eso =0) y mapeo pwm=round(v*255); ViewModel throttling sample(50ms) ⇒ ≤20 msg/s QoS0 a aethernet/rover/command:69 → Gateway CE5/CSN15 Ch76 2 Mbps → Rover CE4/CSN10 ENA5/ENB11 (RF TX 250→249 99.6%). Fail-safe HU-04 500 ms en rover corta PWM si no llega comando. Tests: JoystickMapperTest 10 + ViewModelTest 6 verdes; mosquitto_sub 251 cmds, T3 33k, T4 7.3k verificados.

D. PIN y dashboard (MOV-02/04/08)

LedState.kt + LedStateMapper espejo led.cpp:64/config.h:52 (poll 5 s, ventanas 5s/10s/1s), LedStatusCard 64 dp. PinViewModel throttle 5/60 s + PinScreen 4×3 dots + publishAccessCommand aethernet/access/command:70 → handleAccessCommand:332 → CMD:ACCESS:40 → VALID_PIN 1234:48 → ACCESS:hash 7c78c98f POST 201 + aethernet/access/event:71 (verificado mosquitto pub/sub + curl limit 3, 35 tests). MOV-08 dashboard consolidado (Luces/Rover/Acceso) con navegación inferior y Room offline-first.

Table II
ESTADO MOV AL 2026-09-11 (BACKLOG.MD ÁREA 1)

ID	Tarea	Estado
MOV-01	Setup MVVM base	✓f03190b
MOV-02	LED local solo lectura	✓
MOV-03	MQTT telemetría <50 ms	✓
MOV-04	PIN cerrojo Plan B	✓35 tests
MOV-05/06	Joystick 20 Hz	✓3ca12c9/e821700
MOV-07	BT SPP fallback	Pendiente S3
MOV-08..10	Dashboard + tests	Pendiente S4

IV. RESULTADOS

Latencia percibida <50 ms comando→motor validada con timestamps y contador broker (assembleDebug + testDebugUnitTest verdes, 192.168.1.14:8000/health ok en SM-X620). Capturas en docs/Programacion-Movil/capturas/ (dashboard-led.png, pin-screen.png, joystick-drag.png, mqtt-status.png, rover-card.png) + notebook AetherControl_Notebook.ipynb §capturas.

V. DISCUSIÓN Y LIMITACIONES

La traducción XAML→Compose preserva arquitectura pero exige re-aprender recomposición y remember/mutableStateOf; Paho clásico requiere servicio persistente y manejo explícito de reconexión (mapeado a DashboardUiState). El debounce 20 Hz es compromiso entre fluidez y saturación RF; valores mayores saturan nRF24. Queda habilitar CI android-build (quitar if:false, cache Gradle, subir APK) y cerrar BT SPP (permisos runtime + socket bloqueante).

VI. CONCLUSIONES

AetherControl demuestra que los patrones TS6C3 son transferibles a Kotlin/Compose sin reescribir la didáctica, y que MQTT + MVVM reactivo entrega telemetría y control de baja latencia con trazabilidad RF-1.1/1.2/1.3.

REFERENCES

- [1] UTP, Programa TS6C3 Programación Móvil, 2361_6. TS6C3.pdf.
- [2] AetherNet, Documentación Móvil, docs/Programacion-Movil/programacion-movil.md.
- [3] AetherNet, Roadmap Móvil, docs/Programacion-Movil/roadmap-movil.md.
- [4] AetherNet, Backlog Móvil MOV-01..12, docs/Programacion-Movil/backlog-movil.md.
- [5] AetherNet, Arquitectura §3 topics aethernet/#, docs/architecture.md.
- [6] AetherNet, app MqttManager.kt (Paho 1.2.5) + ServiceLocator.kt:114.
- [7] AetherNet, JoystickScreen.kt + JoystickMapperTest.kt (10 tests).
- [8] AetherNet, PinViewModel/Screen + handleAccessCommand:332 + CMD:ACCESS:40.
- [9] AetherNet, PRD §4 Scope (out iOS), docs/prd.md.
- [10] AetherNet, docs/Programacion-Movil/notebook/AetherControl_Notebook.ipynb.